

halfedge: 细分模块设计文档 (Loop)

src/subdiv.rs (含 catmull_clark、sqrt3 子模块)

halfedge 库

2026-07-04

目录

1	模块定位	2
1.1	核心 API	2
1.2	依赖关系	2
2	算法概述	2
2.1	规模变化	2
2.2	算法流程图	3
3	边点计算	3
3.1	公式	3
3.2	几何图示	4
3.3	实现要点	4
4	顶点更新	4
4.1	内部顶点	4
4.2	边界顶点	4
4.3	邻居识别	5
5	面分裂	5
6	同步更新	5
7	实现代码	5
7.1	核心函数签名	5
7.2	边点计算	6
7.3	β 权重	6
8	健壮性	6
9	复杂度分析	7
10	测试覆盖	7

11 示例输出	8
12 设计取舍	8

1 模块定位

`subdiv.rs` 在 `traversal.rs`、`io.rs` 之上叠加 **曲面细分**能力，是 `halfedge` 库的细分入口。本文件直接实现 **Loop 细分**，并通过两个子模块提供另外两种细分方案：

- `subdiv::catmull_clark`: Catmull-Clark 细分（任意多边形输入，输出三角化的四边形网格），详见 `docs/catmull_clark.tex`；
- `subdiv::sqrt3`: $\sqrt{3}$ 细分（三角网格，面数 $3\times$ 增长），详见 `docs/sqrt3.tex`。

Loop 细分灵感来自 OpenMesh 和 CGAL，用于三角网格的光滑细分。

1.1 核心 API

API	语义
<code>loop_subdivide(&mesh) -> MeshStorage</code>	对三角网格执行一次 Loop 细分，返回新网格
<code>catmull_clark_subdivide(&mesh)</code>	Catmull-Clark 细分（见 <code>catmull_clark</code> 子模块）
<code>sqrt3_subdivide(&mesh)</code>	$\sqrt{3}$ 细分（见 <code>sqrt3</code> 子模块）

1.2 依赖关系

依赖模块	用途
<code>storage</code>	<code>MeshStorage</code> 容器、 <code>Vertex</code> / <code>HalfEdge</code> / <code>Face</code>
<code>traversal</code>	<code>VertexRing</code> 、 <code>FaceHalfEdges</code> 、 <code>is_boundary_*</code>
<code>io</code>	<code>build_mesh_from_vertices_and_faces</code> 构建新网格

2 算法概述

Loop 细分是一种**逼近型**细分方案，每次细分将每个三角面分裂为 4 个小三角形，同时按加权平均更新所有顶点位置，使网格趋于光滑。

2.1 规模变化

设原始网格有 V 顶点、 E 边、 F 面，细分后：

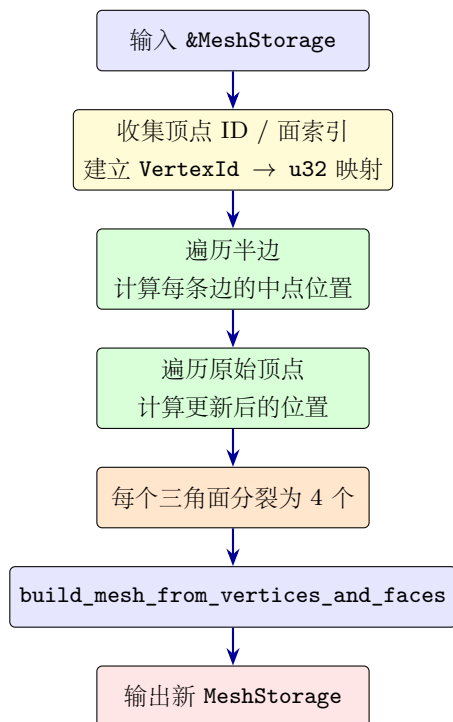
$$V' = V + E, \quad F' = 4F, \quad E' = 4E$$

验证（以 `icosphere(1)` 为例）：

	V	E	F	$V-E+F$	说明
细分前	42	120	80	2	<code>icosphere(1)</code>
细分后	162	480	320	2	$V+E=162, 4F=320$

Euler 示性数 $V - E + F = 2$ 在细分过程中保持不变（闭合三角网格）。

2.2 算法流程图



3 边点计算

3.1 公式

对每条边 (v_0, v_1) ，根据是否为边界边采用不同权重：

内部边（两侧都有面）：

$$v_{\text{new}} = \frac{3}{8}(v_0 + v_1) + \frac{1}{8}(v_2 + v_3)$$

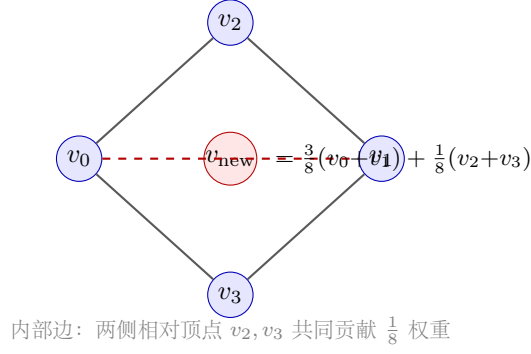
其中 v_2, v_3 是该边两侧相对顶点：

- $v_2 = \text{he.next.vertex}$ (**he** 所在面的对边顶点)
- $v_3 = \text{twin.next.vertex}$ (**twin** 所在面的对边顶点)

边界边（至少一侧无面）：

$$v_{\text{new}} = \frac{1}{2}(v_0 + v_1)$$

3.2 几何图示



3.3 实现要点

- 用 `HashMap<(u32, u32), u32>` 缓存每条无向边的中点索引，键为 $(\min(i_0, i_1), \max(i_0, i_1))$ ，保证每条边只处理一次；
- 遍历所有半边（含边界半边），用 `is_boundary_edge` 判定边类型；
- 内部边若拓扑不完整（`next` 或 `twin.next` 缺失），退化为边界边公式（防御性处理）。

4 顶点更新

4.1 内部顶点

设顶点 v 的 valence 为 n （邻居数），则更新公式：

$$v' = (1 - n\beta)v + \beta \sum_{i=1}^n v_i$$

其中 β 由 Loop 给出：

$$\beta = \frac{1}{n} \left[\frac{5}{8} - \left(\frac{3}{8} + \frac{1}{4} \cos \frac{2\pi}{n} \right)^2 \right]$$

正则情况（ $n = 6$ ）：

$$\cos \frac{2\pi}{6} = \frac{1}{2}, \quad \frac{3}{8} + \frac{1}{4} \cdot \frac{1}{2} = \frac{1}{2}, \quad \beta = \frac{1}{6} \left(\frac{5}{8} - \frac{1}{4} \right) = \frac{1}{16}$$

$$v' = \frac{5}{8}v + \frac{1}{16} \sum_{i=1}^6 v_i$$

非正则情况（ $n = 3$ ）：

$$\cos \frac{2\pi}{3} = -\frac{1}{2}, \quad \beta = \frac{1}{3} \left[\frac{5}{8} - \left(\frac{1}{4} \right)^2 \right] = \frac{3}{16}$$

4.2 边界顶点

边界顶点 v 沿边界方向有两个邻居 $v_{\text{prev}}, v_{\text{next}}$ ，更新公式：

$$v' = \frac{1}{8}v_{\text{prev}} + \frac{3}{4}v + \frac{1}{8}v_{\text{next}}$$

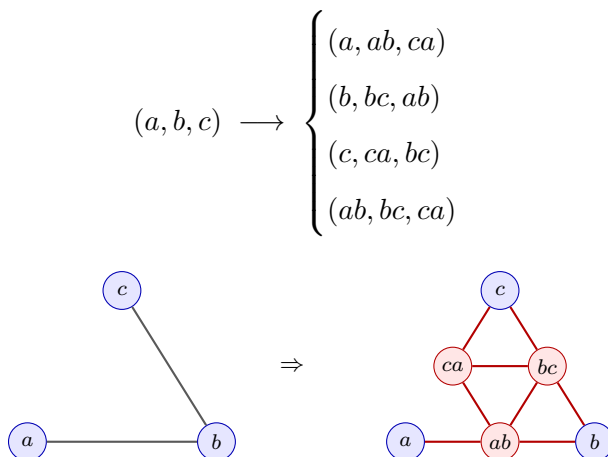
4.3 邻居识别

内部顶点：通过 `VertexRing::new(mesh, v)` 遍历所有 outgoing 半边，每条半边的 `vertex` (tip) 即一个邻居。

边界顶点：同样遍历 outgoing 半边，但用 `is_boundary_edge` 过滤，得到恰好 2 个边界邻居。

5 面分裂

每个原始三角形 (a, b, c) 在三条边中点处插入新顶点 ab, bc, ca ，分裂为 4 个小三角形：



6 同步更新

Loop 细分要求**同步更新**：所有新位置（边中点 + 原始顶点更新）都基于**旧位置**计算，最后一次应用。

本实现的策略：

1. `new_positions` 初始填充原始顶点的旧位置；
2. 步骤 3 计算边中点时，`new_positions[0..V]` 仍为旧值，直接读取；
3. 步骤 4 计算原始顶点新位置时，从 `mesh` 读取旧位置（`mesh` 全程不修改），结果暂存到独立 `updated_orig` 向量；
4. 步骤 4 末尾将 `updated_orig` 覆盖回 `new_positions[0..V]`。

7 实现代码

7.1 核心函数签名

```
1 pub fn loop_subdivide(mesh: &MeshStorage) -> MeshStorage
```

7.2 边点计算

```
1 let midpoint = if is_boundary_edge(mesh, he_id) {
2     //      1/2*(v0+v1)
3     [
4         0.5 * (p0[0] + p1[0]),
5         0.5 * (p0[1] + p1[1]),
6         0.5 * (p0[2] + p1[2]),
7     ]
8 } else {
9     //      3/8*(v0+v1) + 1/8*(v2+v3)
10    let v2 = h.next.and_then(|n| mesh.get_halfedge(n)).map(|n| n.vertex);
11    let v3 = h.twin.and_then(|t| mesh.get_halfedge(t))
12        .and_then(|t| t.next)
13        .and_then(|n| mesh.get_halfedge(n))
14        .map(|n| n.vertex);
15    // ...
16 };
```

7.3 β 权重

```
1 fn loop_beta(n: usize) -> f64 {
2     if n == 0 { return 0.0; } //
3     let n_f = n as f64;
4     let cos_term = (2.0 * std::f64::consts::PI / n_f).cos();
5     let inner = 3.0/8.0 + 1.0/4.0 * cos_term;
6     (1.0 / n_f) * (5.0/8.0 - inner * inner)
7 }
```

8 健壮性

- 无效 / 已删除 ID: 所有查询通过 `mesh.get_* + Option` 链式调用, `None` 时跳过该边 / 该顶点;
- 边界顶点邻居数 $\neq 2$: 保持原位置 (退化情况);
- 内部顶点 `valence = 0`: 保持原位置 (孤立顶点);
- 内部边拓扑不完整 (`next` / `twin.next` 缺失): 退化为边界边公式;
- `loop_beta(0)` 守卫除零, 返回 0;
- 空网格: 返回空网格, 不 panic;
- 非三角面: Loop 细分仅支持三角形。若输入含非三角面 (如四边形), 这些面会被跳过并通过 `eprintln!` 输出警告到 `stderr`, 含跳过面数与替代建议 (`catmull_clark_subdivide`)。三角面照常细分。详见 `loop_subdivide_skips_non_triangle_with_warning` 与 `loop_subdivide_mixed_mesh_k` 测试。

9 复杂度分析

设原始网格有 V 顶点、 E 边、 F 面。

步骤	时间	空间
收集顶点 / 面索引	$O(V + F)$	$O(V + F)$
边点计算（每条边一次）	$O(E)$	$O(E)$ （中点缓存）
顶点更新（每个顶点遍历邻居）	$O(V + E)$	$O(V)$ （新位置数组）
面分裂	$O(F)$	$O(F)$ （新面数组）
构建新网格	$O(V' + E' + F')$	$O(V' + E' + F')$
总计	$O(V + E + F)$	$O(V' + E' + F')$

其中 $V' = V + E$, $E' = 4E$, $F' = 4F$ 。

10 测试覆盖

测试名	验证内容
loop_subdivide_icosphere1_vertex_face_count	icosphere(1) 细分后 $V=162$, $F=320$
loop_subdivide_icosphere0_vertex_face_count	icosphere(0) 细分后 $V=42$, $F=80$
loop_subdivide_icosphere2_vertex_face_count	icosphere(2) 细分后 $V=642$, $F=1280$
loop_subdivide_passes_topology_validation	细分后通过 <code>check_topology</code>
loop_subdivide_icosphere2_passes_validation	icosphere(2) 细分后通过校验
loop_subdivide_preserves_euler_characteristic	Euler 示性数 $V - E + F = 2$ 保持
loop_subdivide_single_triangle	单三角面片细分后 $V=6$, $F=4$
loop_subdivide_open_quad	开四边形细分后 $V=9$, $F=8$
loop_subdivide_boundary_edge_midpoint	边界边中点位置 $= \frac{1}{2}(v_0 + v_1)$
loop_subdivide_interior_edge_midpoint	内部边中点位置 $= \frac{3}{8}(v_0 + v_1) + \frac{1}{8}(v_2 + v_3)$
loop_beta_regular_valence_6	$\beta(6) = \frac{1}{16}$
loop_beta_valence_3	$\beta(3) = \frac{3}{16}$
loop_beta_zero_valence_returns_zero	$\beta(0) = 0$ （不 panic）
loop_subdivide_multiple_iterations_stay_valid	连续 3 次细分拓扑校验通过
loop_subdivide_empty_mesh	空网格细分返回空网格
loop_subdivide_skips_non_triangle_with_warning	单四边形面输入：跳过，输出 0 面（stderr 警告）
loop_subdivide_mixed_mesh_keeps_only_triangles	三角形 + 四边形混合输入：仅三角形细分为 4 面

共 17 个单元测试，全部通过。`cargo test` 全模块共 498 个用例，0 失败 / 0 警告。细分模块（含 `catmull_clark` 11 个与 `sqr3` 16 个子模块测试）合计 44 个测试。

11 示例输出

`cargo run --example loop_subdivision` 的关键输出：

```
[1]      Loop
      icosphere(1): V=   42 F=   80 E=  120 HE=  240
                  : V=  162 F=  320 E=  480 HE=  960
                  V'=V+E (42+120=162) F'=4F (4*80=320)
                  OK

[3]                  icosphere(0) →      1
      5              = 12
      6              = 30
      icosphere(0)   12              =5              =5
                  =6      Loop

[4] Euler          V - E + F
      0x: V-E+F = 12-30+20 = 2      2
      1x: V-E+F = 42-120+80 = 2      2
      2x: V-E+F = 162-480+320 = 2      2
      3x: V-E+F = 642-1920+1280 = 2      2
```

12 设计取舍

- **为何复用 `build_mesh_from_vertices_and_faces`?** 该函数已经处理了 `twin` / `next` / `prev` / 边界环 / 顶点入口 / 面入口的完整构建，避免在本模块重复实现半边拓扑构造逻辑。
- **为何用 `HashMap` 缓存边中点?** 每条无向边只需计算一次中点。用 `(min, max)` 键去重，保证 $O(E)$ 时间 / $O(E)$ 空间。
- **为何同步更新而非渐进更新?** `Loop` 细分要求所有新位置基于旧位置计算。若渐进修改，后续顶点会读到已更新的邻居位置，破坏算法正确性。本实现用独立 `updated_orig` 向量暂存，最后统一覆盖。
- **为何边界顶点邻居数 $\neq 2$ 时保持原位置?** 正常流形网格的边界顶点恰好有 2 个边界邻居。邻居数 $\neq 2$ 说明拓扑非流形或退化，此时不应用边界公式，保持原位置是安全降级。
- **为何不投影回球面?** `Loop` 细分是**逼近型**细分，不保留原始曲面。`test_util` 中 `icosphere` 生成器的细分会把新顶点投影回球面，那是 `icosphere` 的特殊行为，不是 `Loop` 细分本身的职责。